



Already have an account? [Sign in](#) →

# 行程表 (暫定)

| W  | Date         | Lecture                                                                        | Homework           | TA Class |
|----|--------------|--------------------------------------------------------------------------------|--------------------|----------|
| 1  | 09/10, 09/11 | Course Information / No class - Registration                                   | Homework 00 (Wed.) | V        |
| 2  | 09/17, 09/18 | Git Introduction                                                               |                    |          |
| 3  | 09/24, 09/25 | OOP Concept                                                                    | Homework 01 (Wed.) | V        |
| 4  | 10/01, 10/02 | Introducing What C++ is                                                        |                    |          |
| 5  | 10/08, 10/09 | Class / Essential STL Introduction                                             | Homework 02 (Wed.) |          |
| 6  | 10/15, 10/16 | Lec03: Encapsulation                                                           |                    | V        |
| 7  | 10/22, 10/23 | Lec04: Inheritance                                                             | Homework 03 (Wed.) |          |
| 8  | 10/29, 10/30 | Lec04: Inheritance                                                             |                    | V        |
| 9  | 11/05, 11/06 | Physical Hand-Written Midterm                                                  | Homework 04 (Mon.) |          |
| 10 | 11/12, 11/13 | Physical Computer-based Midterm                                                |                    |          |
| 11 | 11/19, 11/20 | Lec05: Polymorphism                                                            | Homework 05 (Wed.) | V        |
| 12 | 11/26, 12/27 | Lec05: Polymorphism                                                            |                    |          |
| 13 | 12/03, 12/04 | Lec05: Polymorphism - LAB                                                      | Homework 06 (Wed.) | V        |
| 14 | 12/10, 12/11 | Lec06: Composition & Interface                                                 |                    |          |
| 15 | 12/17, 12/18 | Lec06: Composition & Interface                                                 | Homework 07 (Wed.) |          |
| 16 | 12/24, 12/25 | Lec06: Composition & Interface - Lab/ No class                                 |                    | V        |
| 17 | 12/31, 01/01 | Physical Hand-Written Final / No class                                         |                    |          |
| 18 | 01/07, 01/08 | Lec07: Experience sharing & OOPL - Preparation / Physical Computer-based Final |                    |          |

# 物件導向程式設計

第肆章：類別

授課講師：孫勤昱博士

國立臺北科技大學 資訊工程系

[cysun@ntut.edu.tw](mailto:cysun@ntut.edu.tw)



# 大綱

- 什麼是類別
- 實作方法
- 生命週期
- RAI



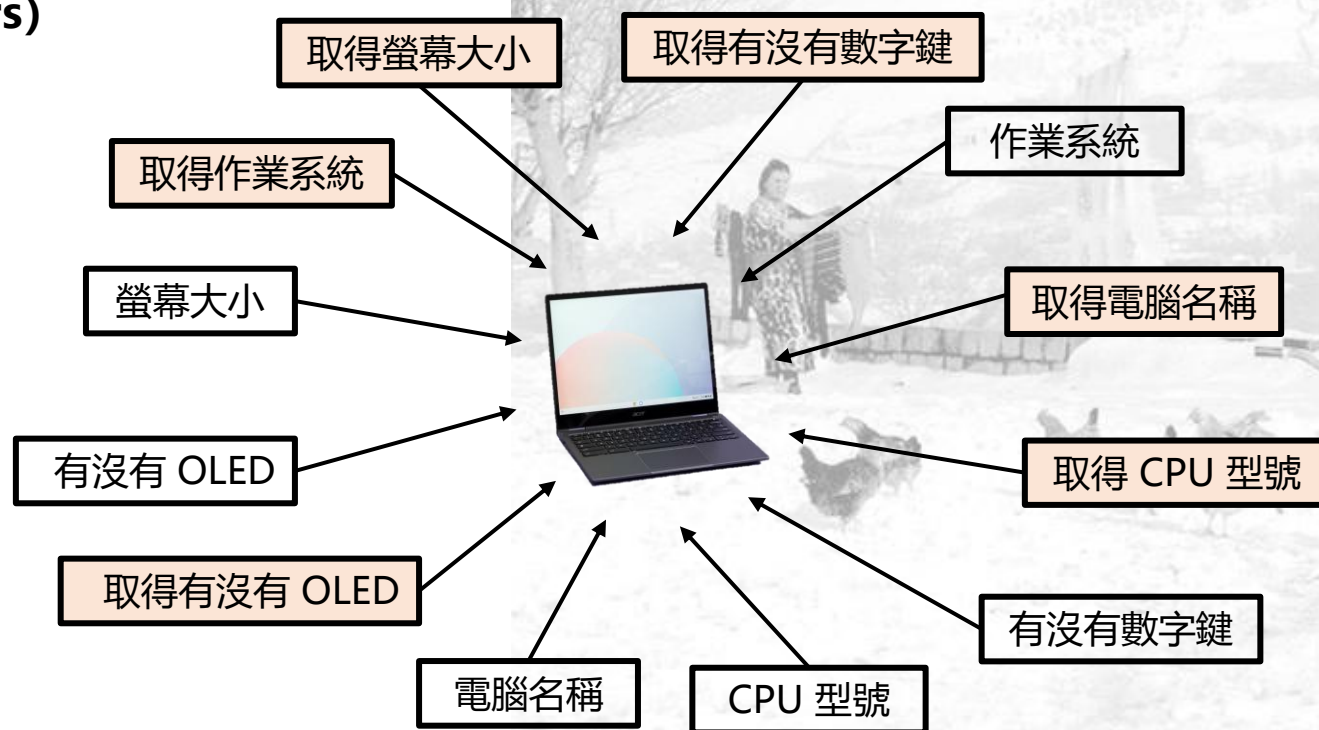
主題一：

# 什麼是類別 (Class)



# 什麼是類別

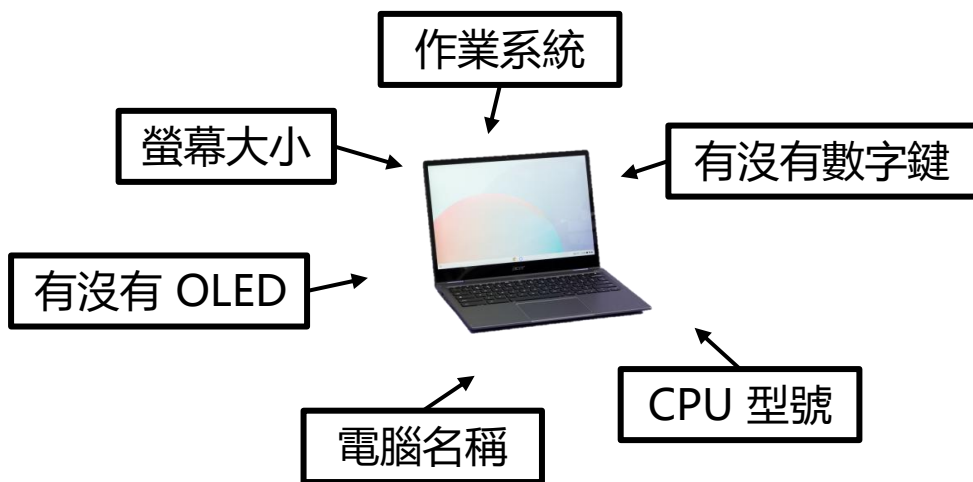
- 用來描述物件 (Object) 的成員 (Members) 與方法 (Methods)
- 成員也稱為屬性 (Attributes) 、狀態 (States)
  - 是類別中的變數，用於儲存與物件相關的資料
- 方法也稱為函數 (Functions) 、行為 (Behaviors)
  - 是類別中，用於執行與物件相關的操作





# 什麼是類別

- 我們可以使用 C++ 的 class 來描述類別
- 例如我們要描述筆電 ...



```
1  #ifndef LAPTOP_HPP
2  #define LAPTOP_HPP
3
4  #include <string>
5
6  class Laptop {
7  public:
8      std::string operatingSystem = "Windows";           // 作業系統 (初始值為 "Windows")
9      double screenSize = 15.7;                          // 螢幕大小 (初始值為 15.7)
10     bool isOLED = false;                                // 是否為 OLED (初始值為 false)
11     std::string name = "Default Laptop";                // 筆電名稱 (初始值為 "Default Laptop")
12     std::string cpuName = "Letni i5-8700";              // CPU 型號 (初始值為 "Letni i5-8700")
13     bool haveNumberKey = false;                         // 是否有數字鍵 (初始值為 false)
14
15     // 建構子，輸入六個值，用於初始化上面的成員
16     Laptop(std::string operatingSystem, double screenSize, bool isOLED,
17           std::string name, std::string cpuName, bool haveNumberKey);
18
19     // 解構子，用來在物件要被釋放時釋放內部資源
20     ~Laptop();
21
22     std::string GetOperatingSystem();                   // 取得作業系統
23     double GetScreenSize();                             // 取得螢幕大小
24     bool IsOLED();                                     // 是否為 OLED
25     std::string GetName();                             // 取得筆電名稱
26     std::string GetCPUName();                          // 取得 CPU 型號
27     bool HaveNumberKey();                             // 是否有數字鍵
28 };
29
30 #endif
31
```

# 指導語句 (Directives)

- 紅框的部分是指導語句 (Directives )
  - 預處理器的指令

```
1  #ifndef LAPTOP_HPP
2  #define LAPTOP_HPP
3
4  #include <string>
5
6  class Laptop {
7  public:
8      std::string operatingSystem = "Windows";           // 作業系統 (初始值為 "Windows")
9      double screenSize = 15.7;                          // 螢幕大小 (初始值為 15.7)
10     bool isOLED = false;                                // 是否為 OLED (初始值為 false)
11     std::string name = "Default Laptop";                // 筆電名稱 (初始值為 "Default Laptop")
12     std::string cpuName = "Letni i5-8700";               // CPU 型號 (初始值為 "Letni i5-8700")
13     bool haveNumberKey = false;                         // 是否有數字鍵 (初始值為 false)
14
15     // 建構子，輸入六個值，用於初始化上面的成員
16     Laptop(std::string operatingSystem, double screenSize, bool isOLED,
17           std::string name, std::string cpuName, bool haveNumberKey);
18
19     // 解構子，用來在物件要被釋放時釋放內部資源
20     ~Laptop();
21
22     std::string GetOperatingSystem();                    // 取得作業系統
23     double GetScreenSize();                             // 取得螢幕大小
24     bool IsOLED();                                       // 是否為 OLED
25     std::string GetName();                              // 取得筆電名稱
26     std::string GetCPUName();                           // 取得 CPU 型號
27     bool HaveNumberKey();                               // 是否有數字鍵
28 };
29
30 #endif
31
```

# Include Guards

- `#ifndef LAPTOP_HPP`: 這個指令會檢查是否已經定義過 `LAPTOP_HPP`
  - 如果還未定義，則預處理器會繼續執行接下來的code
- `#define LAPTOP_HPP`: 這個指令定義了一個 `LAPTOP_HPP`
  - 這樣做的目的是為了讓預處理器「記住」這個頭文件已經被包含過了，以避免未來重複包含
- `#endif`: 這個指令表示 `#ifndef` 條件的結束

```
1  #ifndef LAPTOP_HPP
2  #define LAPTOP_HPP
3
4  #include <string>
5
6  class Laptop {
7  public:
8      std::string operatingSystem = "Windows";    // 作業系統 (初始值為 "Windows")
9      double screenSize = 15.7;                  // 螢幕大小 (初始值為 15.7)
10     bool isOLED = false;                        // 是否為 OLED (初始值為 false)
11     std::string name = "Default Laptop";        // 筆電名稱 (初始值為 "Default Laptop")
12     std::string cpuName = "Letni i5-8700";      // CPU 型號 (初始值為 "Letni i5-8700")
13     bool haveNumberKey = false;                // 是否有數字鍵 (初始值為 false)
14
15     // 建構子，輸入六個值，用於初始化上面的成員
16     Laptop(std::string operatingSystem, double screenSize, bool isOLED,
17            std::string name, std::string cpuName, bool haveNumberKey);
18
19     // 解構子，用來在物件要被釋放時釋放內部資源
20     ~Laptop();
21
22     std::string GetOperatingSystem();            // 取得作業系統
23     double GetScreenSize();                     // 取得螢幕大小
24     bool IsOLED();                             // 是否為 OLED
25     std::string GetName();                     // 取得筆電名稱
26     std::string GetCPUName();                  // 取得 CPU 型號
27     bool HaveNumberKey();                      // 是否有數字鍵
28 };
29
30 #endif
31
```



# 類別名稱

- 紅框的部分是類別名稱 (Class Name)
  - 用來當作這個類別 (Class) 的名稱

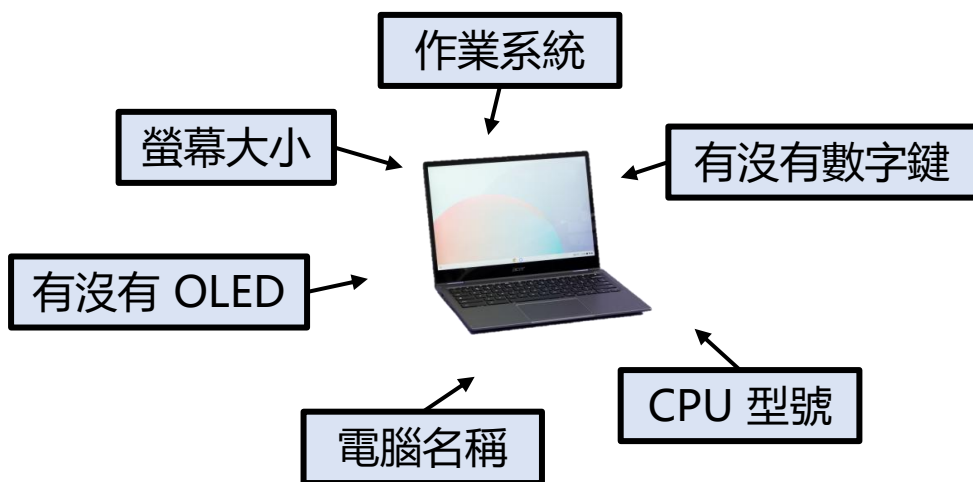


Laptop

```
1  #ifndef LAPTOP_HPP
2  #define LAPTOP_HPP
3
4  #include <string>
5
6  class Laptop {
7  public:
8      std::string operatingSystem = "Windows";           // 作業系統 (初始值為 "Windows")
9      double screenSize = 15.7;                          // 螢幕大小 (初始值為 15.7)
10     bool isOLED = false;                                // 是否為 OLED (初始值為 false)
11     std::string name = "Default Laptop";                // 筆電名稱 (初始值為 "Default Laptop")
12     std::string cpuName = "Letni i5-8700";              // CPU 型號 (初始值為 "Letni i5-8700")
13     bool haveNumberKey = false;                         // 是否有數字鍵 (初始值為 false)
14
15     // 建構子，輸入六個值，用於初始化上面的成員
16     Laptop(std::string operatingSystem, double screenSize, bool isOLED,
17           std::string name, std::string cpuName, bool haveNumberKey);
18
19     // 解構子，用來在物件要被釋放時釋放內部資源
20     ~Laptop();
21
22     std::string GetOperatingSystem();                   // 取得作業系統
23     double GetScreenSize();                             // 取得螢幕大小
24     bool IsOLED();                                     // 是否為 OLED
25     std::string GetName();                             // 取得筆電名稱
26     std::string GetCPUName();                          // 取得 CPU 型號
27     bool HaveNumberKey();                              // 是否有數字鍵
28 };
29
30 #endif
31
```

# 類別成員

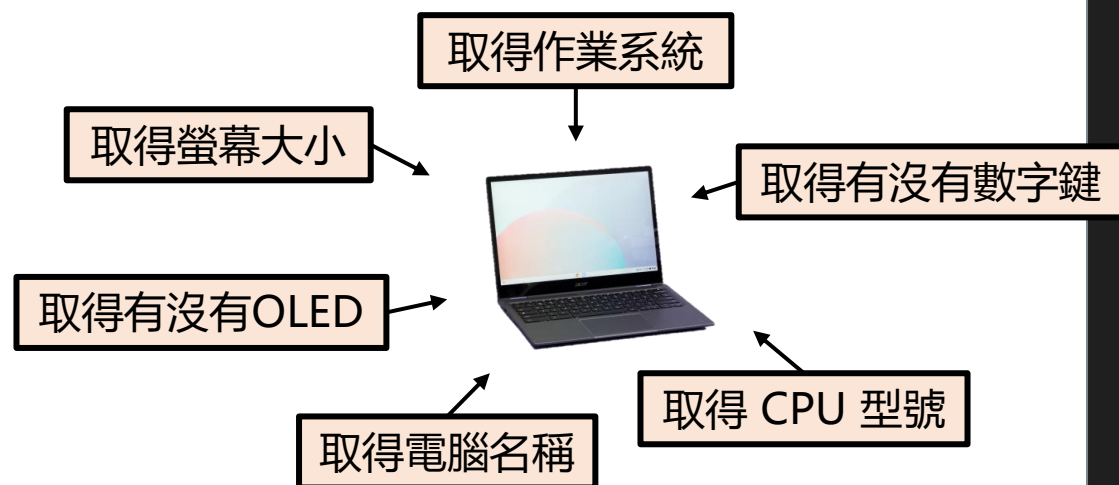
- 紅框的部分是類別**成員** (Class Member)
  - 用來描述這個類別所擁有的成員



```
1  #ifndef LAPTOP_HPP
2  #define LAPTOP_HPP
3
4  #include <string>
5
6  class Laptop {
7  public:
8      std::string operatingSystem = "Windows";           // 作業系統 (初始值為 "Windows")
9      double screenSize = 15.7;                          // 螢幕大小 (初始值為 15.7)
10     bool isOLED = false;                                // 是否為 OLED (初始值為 false)
11     std::string name = "Default Laptop";                // 筆電名稱 (初始值為 "Default Laptop")
12     std::string cpuName = "Letni i5-8700";              // CPU 型號 (初始值為 "Letni i5-8700")
13     bool haveNumberKey = false;                         // 是否有數字鍵 (初始值為 false)
14
15     // 建構子，輸入六個值，用於初始化上面的成員
16     Laptop(std::string operatingSystem, double screenSize, bool isOLED,
17           std::string name, std::string cpuName, bool haveNumberKey);
18
19     // 解構子，用來在物件要被釋放時釋放內部資源
20     ~Laptop();
21
22     std::string GetOpeartingSystem();                   // 取得作業系統
23     double GetScreenSize();                             // 取得螢幕大小
24     bool IsOLED();                                     // 是否為 OLED
25     std::string GetName();                             // 取得筆電名稱
26     std::string GetCPUName();                          // 取得 CPU 型號
27     bool HaveNumberKey();                              // 是否有數字鍵
28 };
29
30 #endif
31
```

# 類別方法

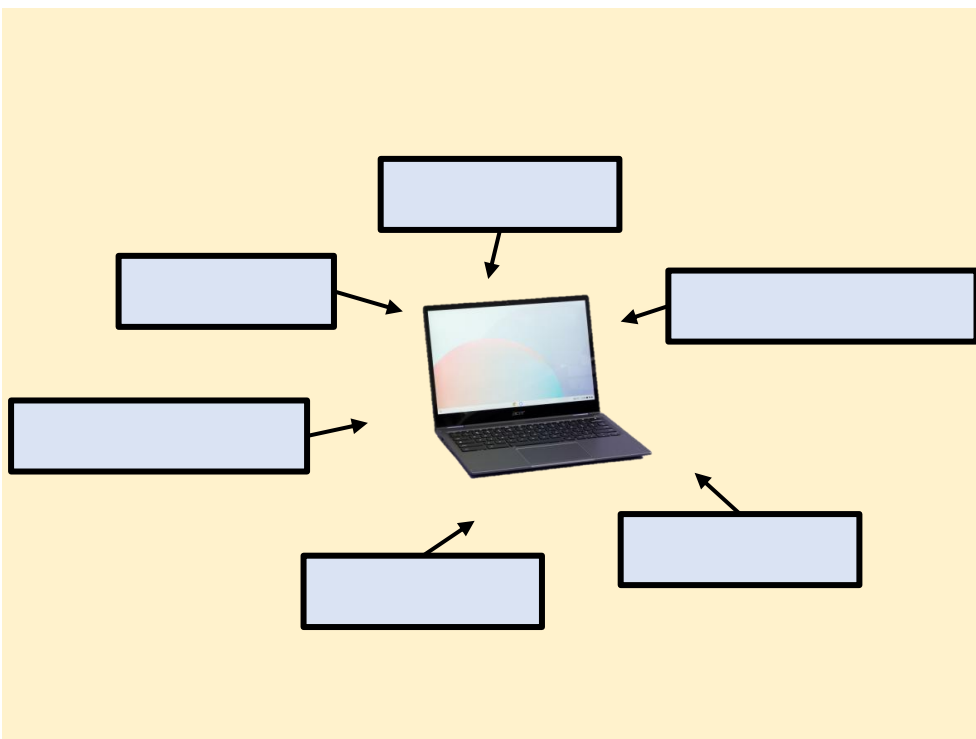
- 紅框的部分是類別**方法** (Class Method)
  - 用來描述這個類別所擁有的方法



```
1  #ifndef LAPTOP_HPP
2  #define LAPTOP_HPP
3
4  #include <string>
5
6  class Laptop {
7  public:
8      std::string operatingSystem = "Windows"; // 作業系統 (初始值為 "Windows")
9      double screenSize = 15.7; // 螢幕大小 (初始值為 15.7)
10     bool isOLED = false; // 是否為 OLED (初始值為 false)
11     std::string name = "Default Laptop"; // 筆電名稱 (初始值為 "Default Laptop")
12     std::string cpuName = "Letni i5-8700"; // CPU 型號 (初始值為 "Letni i5-8700")
13     bool haveNumberKey = false; // 是否有數字鍵 (初始值為 false)
14
15     // 建構子，輸入六個值，用於初始化上面的成員
16     Laptop(std::string operatingSystem, double screenSize, bool isOLED,
17           std::string name, std::string cpuName, bool haveNumberKey);
18
19     // 解構子，用來在物件要被釋放時釋放內部資源
20     ~Laptop();
21
22     std::string GetOpeartingSystem(); // 取得作業系統
23     double GetScreenSize(); // 取得螢幕大小
24     bool IsOLED(); // 是否為 OLED
25     std::string GetName(); // 取得筆電名稱
26     std::string GetCPUName(); // 取得 CPU 型號
27     bool HaveNumberKey(); // 是否有數字鍵
28 };
29
30 #endif
31
```

# 建構子 (1/2)

- 我們使用**建構子**來初始化類別的成員

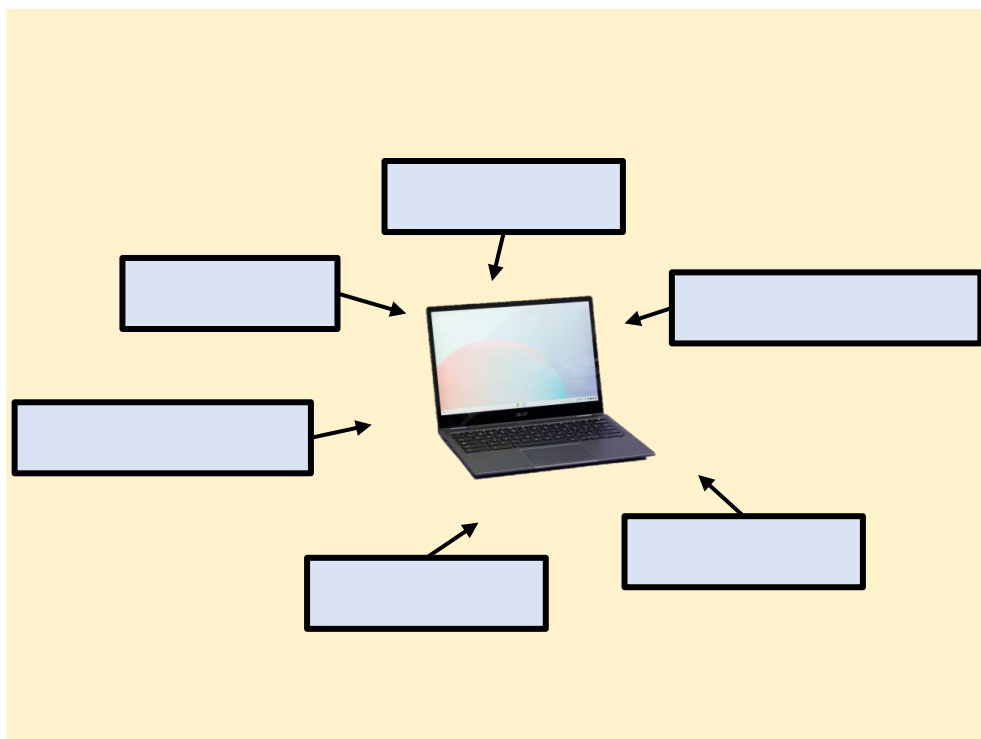


```
1  #ifndef LAPTOP_HPP
2  #define LAPTOP_HPP
3
4  #include <string>
5
6  class Laptop {
7  public:
8      std::string operatingSystem = "Windows";           // 作業系統 (初始值為 "Windows")
9      double screenSize = 15.7;                          // 螢幕大小 (初始值為 15.7)
10     bool isOLED = false;                                // 是否為 OLED (初始值為 false)
11     std::string name = "Default Laptop";                // 筆電名稱 (初始值為 "Default Laptop")
12     std::string cpuName = "Letni i5-8700";              // CPU 型號 (初始值為 "Letni i5-8700")
13     bool haveNumberKey = false;                         // 是否有數字鍵 (初始值為 false)
14
15     // 建構子，輸入六個值，用於初始化上面的成員
16     Laptop(std::string operatingSystem, double screenSize, bool isOLED,
17            std::string name, std::string cpuName, bool haveNumberKey);
18
19     // 解構子，用來在物件要被釋放時釋放內部資源
20     ~Laptop();
21
22     std::string GetOpeartingSystem();                   // 取得作業系統
23     double GetScreenSize();                             // 取得螢幕大小
24     bool IsOLED();                                       // 是否為 OLED
25     std::string GetName();                               // 取得筆電名稱
26     std::string GetCPUName();                           // 取得 CPU 型號
27     bool HaveNumberKey();                               // 是否有數字鍵
28 };
29
30 #endif
31
```

# 建構子 (2/2)

- 可以透過建構子初始化成員，並建立物件

```
1 #include "laptop.hpp"
2
3 int main(){
4     Laptop myLaptop = Laptop("Windows", 15.9, true, "My Laptop", "Letni i9-48763", true);
5 }
```



```
1 #ifndef LAPTOP_HPP
2 #define LAPTOP_HPP
3
4 #include <string>
5
6 class Laptop {
7 public:
8     std::string operatingSystem = "Windows"; // 作業系統 (初始值為 "Windows")
9     double screenSize = 15.7; // 螢幕大小 (初始值為 15.7)
10    bool isOLED = false; // 是否為 OLED (初始值為 false)
11    std::string name = "Default Laptop"; // 筆電名稱 (初始值為 "Default Laptop")
12    std::string cpuName = "Letni i5-8700"; // CPU 型號 (初始值為 "Letni i5-8700")
13    bool haveNumberKey = false; // 是否有數字鍵 (初始值為 false)
14
15    // 建構子，輸入六個值，用於初始化上面的成員
16    Laptop(std::string operatingSystem, double screenSize, bool isOLED,
17          std::string name, std::string cpuName, bool haveNumberKey);
18
19    // 解構子，用來在物件要被釋放時釋放內部資源
20    ~Laptop();
21
22    std::string GetOpeartingSystem(); // 取得作業系統
23    double GetScreenSize(); // 取得螢幕大小
24    bool IsOLED(); // 是否為 OLED
25    std::string GetName(); // 取得筆電名稱
26    std::string GetCPUName(); // 取得 CPU 型號
27    bool HaveNumberKey(); // 是否有數字鍵
28 };
29
30 #endif
31
```



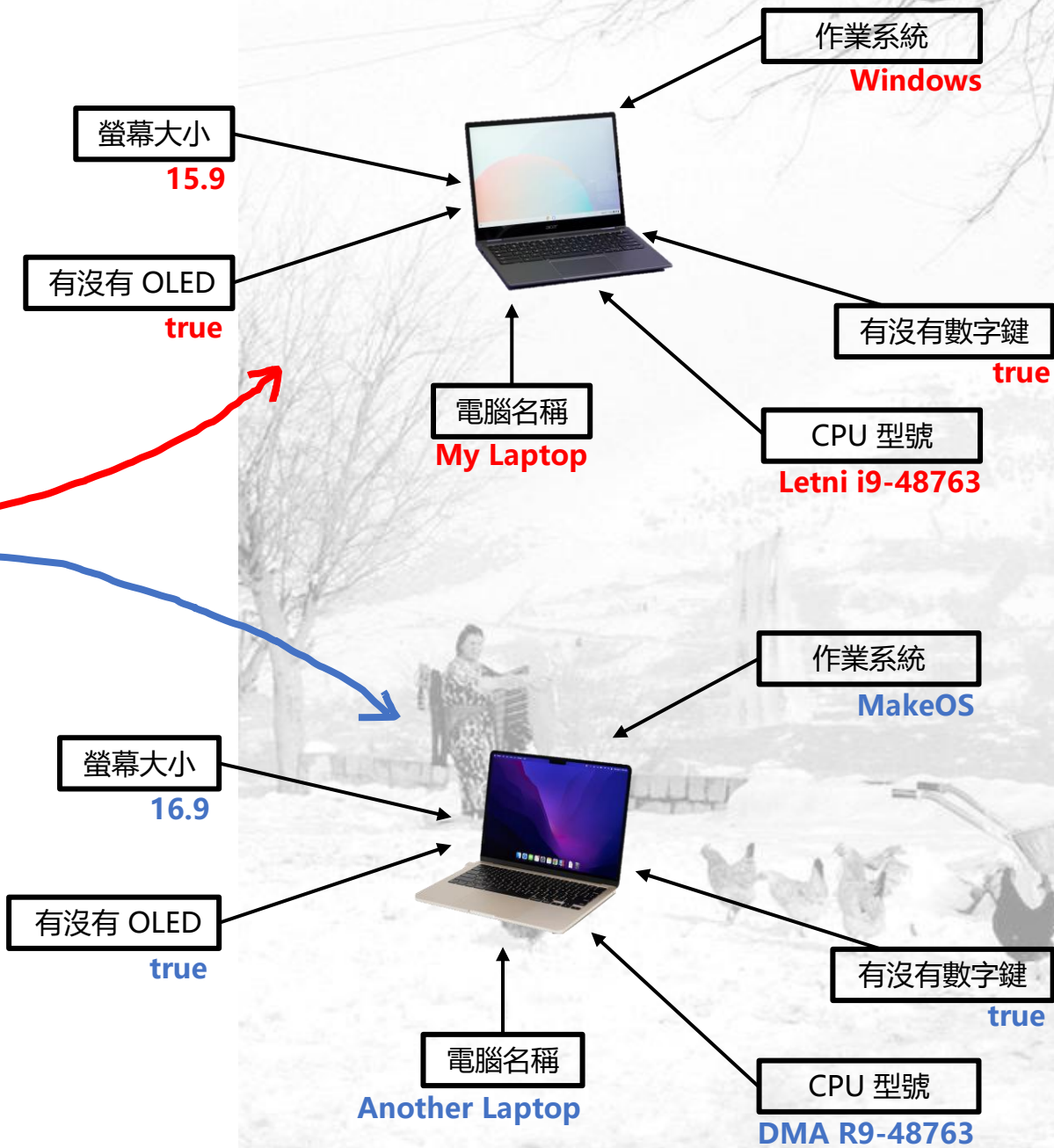
# 建立物件

- 然後就能用類別來建立出很多台電腦

```
1 #include "laptop.hpp"
2
3 int main(){
4     Laptop myLaptop = Laptop("Windows", 15.9, true, "My Laptop", "Letni i9-48763", true);
5     Laptop anotherLaptop = Laptop("MakeOS", 16.9, true, "Another Laptop", "DMA R9-48763", true);
6 }
```

- 並且可透過類別名稱，來取得物件相關的屬性

```
1 #include "laptop.hpp"
2
3 int main(){
4     Laptop myLaptop = Laptop("Windows", 15.9, true, "My Laptop", "Letni i9-48763", true);
5     Laptop anotherLaptop = Laptop("MakeOS", 16.9, true, "Another Laptop", "DMA R9-48763", true);
6
7     std::string myLaptopCPUName = myLaptop.cpuName;           // My Laptop
8     std::string anotherLaptopOperatingSystem = anotherLaptop.operatingSystem; // MakeOS
9 }
```



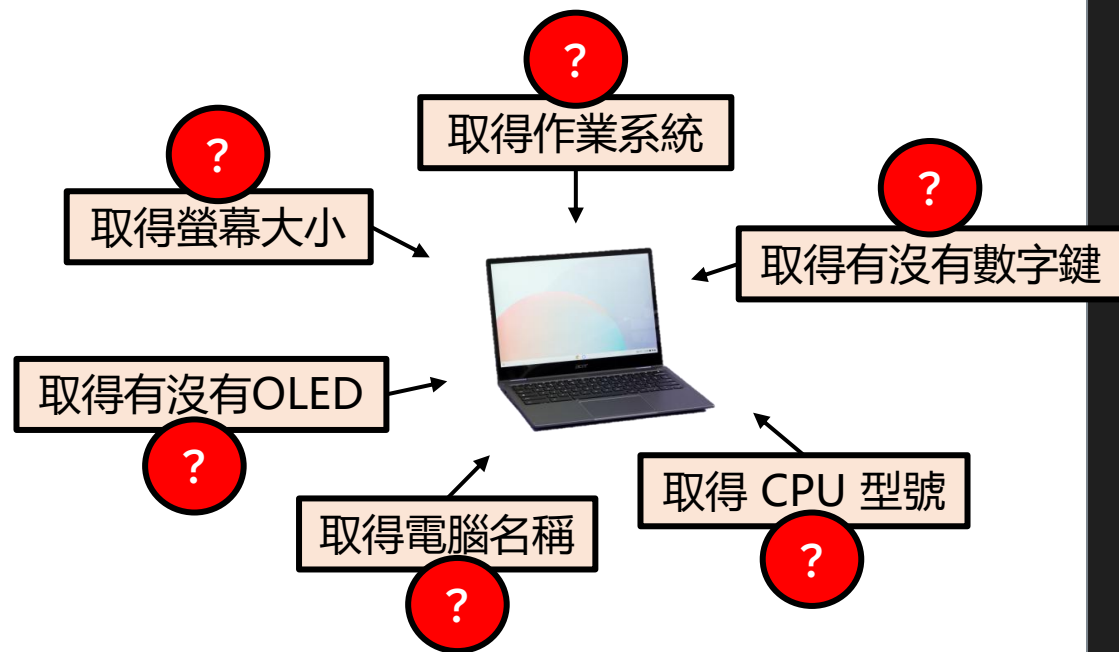
主題二：

# 實作方法 (Method)



# 實作方法

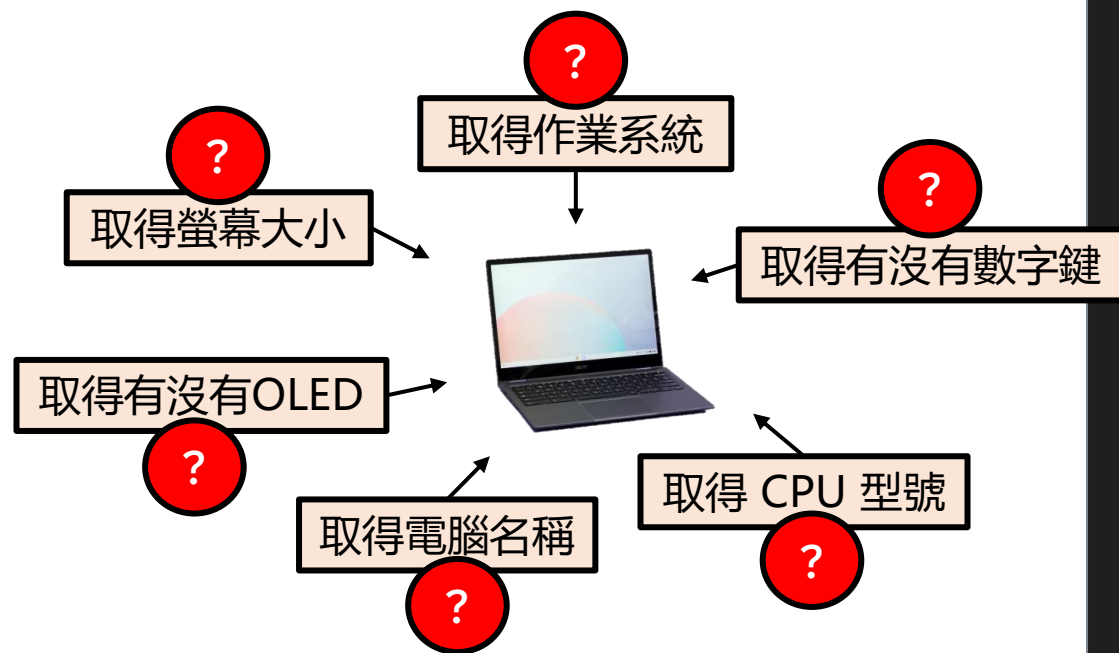
- 但電腦不會知道怎麼「取得作業系統」
- 所以我們需要實作方法的細節來讓電腦知道



```
1  #ifndef LAPTOP_HPP
2  #define LAPTOP_HPP
3
4  #include <string>
5
6  class Laptop {
7  public:
8      std::string operatingSystem = "Windows"; // 作業系統 (初始值為 "Windows")
9      double screenSize = 15.7; // 螢幕大小 (初始值為 15.7)
10     bool isOLED = false; // 是否為 OLED (初始值為 false)
11     std::string name = "Default Laptop"; // 筆電名稱 (初始值為 "Default Laptop")
12     std::string cpuName = "Letni i5-8700"; // CPU 型號 (初始值為 "Letni i5-8700")
13     bool haveNumberKey = false; // 是否有數字鍵 (初始值為 false)
14
15     // 建構子，輸入六個值，用於初始化上面的成員
16     Laptop(std::string operatingSystem, double screenSize, bool isOLED,
17           std::string name, std::string cpuName, bool haveNumberKey);
18
19     // 解構子，用來在物件要被釋放時釋放內部資源
20     ~Laptop();
21
22     std::string GetOpeartingSystem(); // 取得作業系統
23     double GetScreenSize(); // 取得螢幕大小
24     bool IsOLED(); // 是否為 OLED
25     std::string GetName(); // 取得筆電名稱
26     std::string GetCPUName(); // 取得 CPU 型號
27     bool HaveNumberKey(); // 是否有數字鍵
28 };
29
30 #endif
31
```

# 實作方法

- 在設計類別時，我們已經定義了方法



```
1  #ifndef LAPTOP_HPP
2  #define LAPTOP_HPP
3
4  #include <string>
5
6  class Laptop {
7  public:
8      std::string operatingSystem = "Windows"; // 作業系統 (初始值為 "Windows")
9      double screenSize = 15.7; // 螢幕大小 (初始值為 15.7)
10     bool isOLED = false; // 是否為 OLED (初始值為 false)
11     std::string name = "Default Laptop"; // 筆電名稱 (初始值為 "Default Laptop")
12     std::string cpuName = "Letni i5-8700"; // CPU 型號 (初始值為 "Letni i5-8700")
13     bool haveNumberKey = false; // 是否有數字鍵 (初始值為 false)
14
15     // 建構子，輸入六個值，用於初始化上面的成員
16     Laptop(std::string operatingSystem, double screenSize, bool isOLED,
17           std::string name, std::string cpuName, bool haveNumberKey);
18
19     // 解構子，用來在物件要被釋放時釋放內部資源
20     ~Laptop();
21
22     std::string GetOpeartingSystem(); // 取得作業系統
23     double GetScreenSize(); // 取得螢幕大小
24     bool IsOLED(); // 是否為 OLED
25     std::string GetName(); // 取得筆電名稱
26     std::string GetCPUName(); // 取得 CPU 型號
27     bool HaveNumberKey(); // 是否有數字鍵
28 };
29
30 #endif
31
```

# 實作方法

- 因此我們需要實作函數的細節

```
.hpp
1 #ifndef LAPTOP_HPP
2 #define LAPTOP_HPP
3
4 #include <string>
5
6 class Laptop {
7 public:
8     std::string operatingSystem = "Windows";    // 作業系統 (初始值為 "Windows")
9     double screenSize = 15.7;                  // 螢幕大小 (初始值為 15.7)
10    bool isOLED = false;                        // 是否為 OLED (初始值為 false)
11    std::string name = "Default Laptop";        // 筆電名稱 (初始值為 "Default Laptop")
12    std::string cpuName = "Letni i5-8700";      // CPU 型號 (初始值為 "Letni i5-8700")
13    bool haveNumberKey = false;                 // 是否有數字鍵 (初始值為 false)
14
15    // 建構子，輸入六個值，用於初始化上面的成員
16    Laptop(std::string operatingSystem, double screenSize, bool isOLED,
17          std::string name, std::string cpuName, bool haveNumberKey);
18
19    // 解構子，用來在物件要被釋放時釋放內部資源
20    ~Laptop();
21
22    std::string GetOperatingSystem();            // 取得作業系統
23    double GetScreenSize();                     // 取得螢幕大小
24    bool IsOLED();                              // 是否為 OLED
25    std::string GetName();                      // 取得筆電名稱
26    std::string GetCPUName();                   // 取得 CPU 型號
27    bool HaveNumberKey();                       // 是否有數字鍵
28 };
29
30 #endif
31
```

```
.cpp
1 #include "laptop.hpp"
2
3 #include <string>
4
5 Laptop::Laptop(std::string operatingSystem, double screenSize, bool isOLED,
6               std::string name, std::string cpuName, bool haveNumberKey){
7     this->operatingSystem = operatingSystem;
8     this->screenSize = screenSize;
9     this->isOLED = isOLED;
10    this->name = name;
11    this->cpuName = cpuName;
12    this->haveNumberKey = haveNumberKey;
13 }
14
15 Laptop::~~Laptop(){
16 }
17
18
19 std::string Laptop::GetOperatingSystem(){
20     return operatingSystem;
21 }
22
23 double Laptop::GetScreenSize(){
24     return screenSize;
25 }
26
27 bool Laptop::IsOLED(){
28     return isOLED;
29 }
30
31 std::string Laptop::GetName(){
32     return name;
33 }
34
35 std::string Laptop::GetCPUName(){
36     return cpuName;
37 }
38
39 bool Laptop::HaveNumberKey(){
40     return haveNumberKey;
41 }
```



# 實作方法

- 當我們完成了方法的定義後，就能夠呼叫方法。

```
1 #ifndef LAPTOP_HPP
2 #define LAPTOP_HPP
3
4 #include <string>
5
6 class Laptop {
7 public:
8     std::string operatingSystem = "Windows";    // 作業系統 (初始值為 "Windows")
9     double screenSize = 15.7;                  // 螢幕大小 (初始值為 15.7)
10    bool isOLED = false;                        // 是否為 OLED (初始值為 false)
11    std::string name = "Default Laptop";        // 筆電名稱 (初始值為 "Default Laptop")
12    std::string cpuName = "Letni i5-8700";      // CPU 型號 (初始值為 "Letni i5-8700")
13    bool haveNumberKey = false;                // 是否有數字鍵 (初始值為 false)
14
15    // 建構子，輸入六個值，用於初始化上面的成員
16    Laptop(std::string operatingSystem, double screenSize, bool isOLED,
17           std::string name, std::string cpuName, bool haveNumberKey);
18
19    // 解構子，用來在物件要被釋放時釋放內部資源
20    ~Laptop();
21
22    std::string GetOperatingSystem();            // 取得作業系統
23    double GetScreenSize();                     // 取得螢幕大小
24    bool IsOLED();                              // 是否為 OLED
25    std::string GetName();                      // 取得筆電名稱
26    std::string GetCPUName();                   // 取得 CPU 型號
27    bool HaveNumberKey();                       // 是否有數字鍵
28 };
29
30 #endif
```

1: Constructor declaration in the class definition.

2: `GetOperatingSystem()` declaration in the class definition.

3: `GetScreenSize()` declaration in the class definition.

4: `IsOLED()` declaration in the class definition.

5: `GetName()` declaration in the class definition.

6: `GetCPUName()` declaration in the class definition.

```
1 #include "laptop.hpp"
2
3 #include <string>
4
5 Laptop::Laptop(std::string operatingSystem, double screenSize, bool isOLED,
6               std::string name, std::string cpuName, bool haveNumberKey){
7     this->operatingSystem = operatingSystem;
8     this->screenSize = screenSize;
9     this->isOLED = isOLED;
10    this->name = name;
11    this->cpuName = cpuName;
12    this->haveNumberKey = haveNumberKey;
13 }
14
15 Laptop::~Laptop(){
16 }
17
18
19 std::string Laptop::GetOperatingSystem(){
20     return operatingSystem;
21 }
22
23 double Laptop::GetScreenSize(){
24     return screenSize;
25 }
26
27 bool Laptop::IsOLED(){
28     return isOLED;
29 }
30
31 std::string Laptop::GetName(){
32     return name;
33 }
34
35 std::string Laptop::GetCPUName(){
36     return cpuName;
37 }
38
39 bool Laptop::HaveNumberKey(){
40     return haveNumberKey;
41 }
```



主題三：

# This 指標

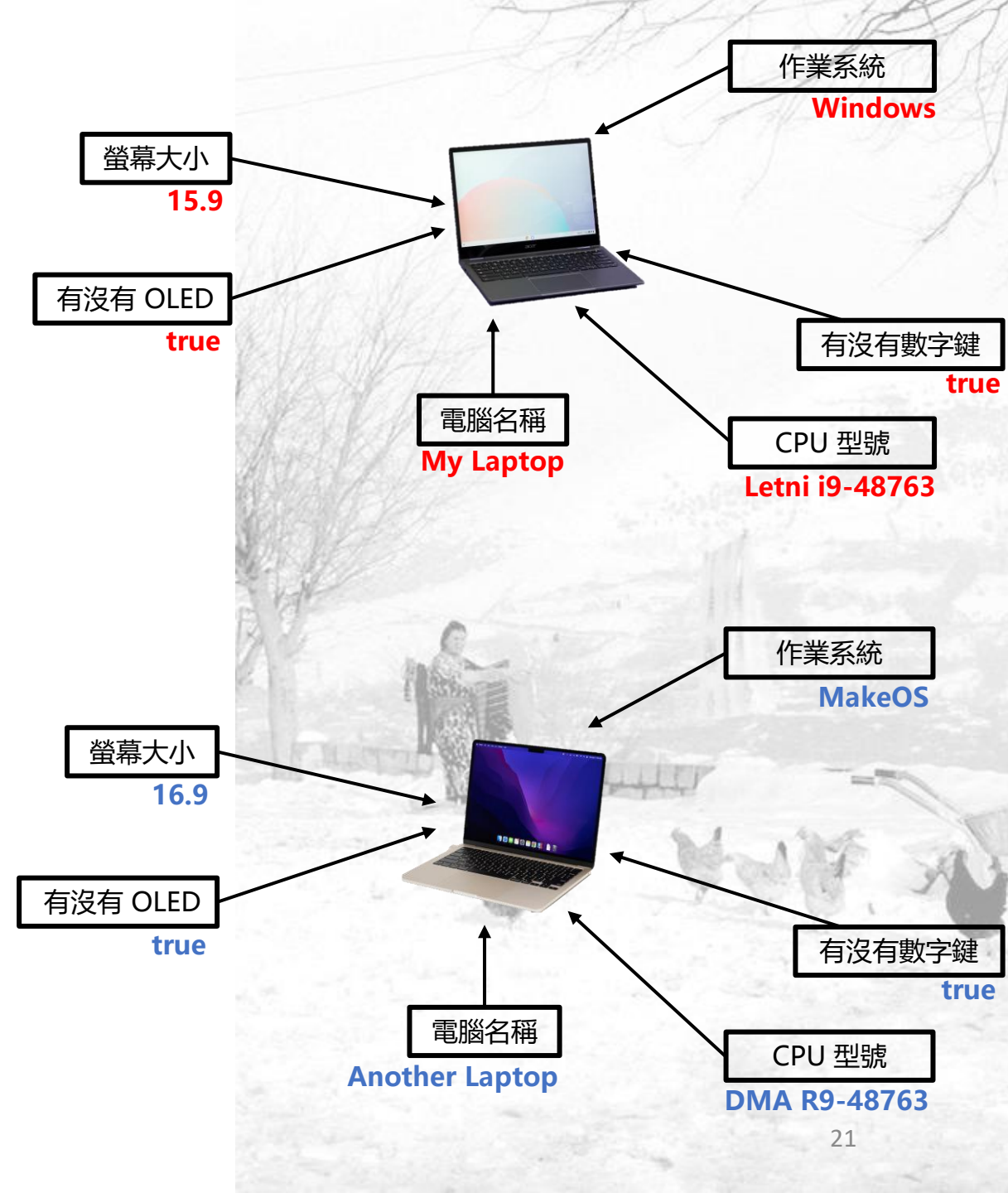


# 問題1

- 當我們在主程式中建立兩台筆電時
- 每個物件都有自己的資料

```
1 #include "laptop.hpp"
2
3 int main(){
4     Laptop myLaptop = Laptop("Windows", 15.9, true, "My Laptop", "Letni i9-48763", true);
5     Laptop anotherLaptop = Laptop("MakeOS", 16.9, true, "Another Laptop", "DMA R9-48763", true);
6
7     std::string myLaptopCPUName = myLaptop.cpuName;           // My Laptop
8     std::string anotherLaptopOperatingSystem = anotherLaptop.operatingSystem; // MakeOS
9 }
```

- 當呼叫 myLaptop.cpuName 時候，程式要怎麼知道要印出 "Letni i9-48763" 而不是 "DMA-R948763" 呢？



# 說明

- 靠 **this** 指標
- C++ 會偷偷幫你加上一個「指標」，叫做 **this**，它指向目前正在被操作的物件
- 要存取裡面的成員就要用箭頭 ->
- 實際上程式碼如下：
- `std::string Laptop::GetCPUName(){`
- `return this->cpuName;`
- `}`

```
.cpp
1  #include "laptop.hpp"
2
3  #include <string>
4
5  Laptop::Laptop(std::string operatingSystem, double screenSize, bool isOLED,
6               std::string name, std::string cpuName, bool haveNumberKey){
7      this->operatingSystem = operatingSystem;
8      this->screenSize = screenSize;
9      this->isOLED = isOLED;
10     this->name = name;
11     this->cpuName = cpuName;
12     this->haveNumberKey = haveNumberKey;
13 }
14
15 Laptop::~~Laptop(){
16 }
17
18
19 std::string Laptop::GetOperatingSystem(){
20     return operatingSystem;
21 }
22
23 double Laptop::GetScreenSize(){
24     return screenSize;
25 }
26
27 bool Laptop::IsOLED(){
28     return isOLED;
29 }
30
31 std::string Laptop::GetName(){
32     return name;
33 }
34
35 std::string Laptop::GetCPUName(){
36     return cpuName;
37 }
38
39 bool Laptop::HaveNumberKey(){
40     return haveNumberKey;
41 }
```

# 問題2

- 有加跟沒加差在哪？

```
.cpp
1  #include "laptop.hpp"
2
3  #include <string>
4
5  Laptop::Laptop(std::string operatingSystem, double screenSize, bool isOLED,
6               std::string name, std::string cpuName, bool haveNumberKey){
7      this->operatingSystem = operatingSystem;
8      this->screenSize = screenSize;
9      this->isOLED = isOLED;
10     this->name = name;
11     this->cpuName = cpuName;
12     this->haveNumberKey = haveNumberKey;
13 }
14
15 Laptop::~~Laptop(){
16 }
17
18
19 std::string Laptop::GetOpeartingSystem(){
20     return operatingSystem;
21 }
22
23 double Laptop::GetScreenSize(){
24     return screenSize;
25 }
26
27 bool Laptop::IsOLED(){
28     return isOLED;
29 }
30
31 std::string Laptop::GetName(){
32     return name;
33 }
34
35 std::string Laptop::GetCPUName(){
36     return cpuName;
37 }
38
39 bool Laptop::HaveNumberKey(){
40     return haveNumberKey;
41 }
```

# 說明

- 左邊: `this->operatingSystem` → 物件裡的成員變數
- 右邊: `operatingSystem` → 傳進來的參數
- 如果不加 `this->`
  - C++ 會以為你在「把參數自己賦值給自己」
  - 完全沒改變任何東西
- `this -> cpuName` 意思是我這台筆電cpu的名字
- `cpuName` 意思是傳進來的cpu名字
- `this-> cpuName = cpuName` 意思是把傳進來的名字存入我這台筆電的cpu名字中
- 如果變數名稱不衝突, 就可以省略 `this->`

```
.cpp
1  #include "laptop.hpp"
2
3  #include <string>
4
5  Laptop::Laptop(std::string operatingSystem, double screenSize, bool isOLED,
6               std::string name, std::string cpuName, bool haveNumberKey){
7      this->operatingSystem = operatingSystem;
8      this->screenSize = screenSize;
9      this->isOLED = isOLED;
10     this->name = name;
11     this->cpuName = cpuName;
12     this->haveNumberKey = haveNumberKey;
13 }
14
15 Laptop::~Laptop(){
16 }
17
18
19 std::string Laptop::GetOperatingSystem(){
20     return operatingSystem;
21 }
22
23 double Laptop::GetScreenSize(){
24     return screenSize;
25 }
26
27 bool Laptop::IsOLED(){
28     return isOLED;
29 }
30
31 std::string Laptop::GetName(){
32     return name;
33 }
34
35 std::string Laptop::GetCPUName(){
36     return cpuName;
37 }
38
39 bool Laptop::HaveNumberKey(){
40     return haveNumberKey;
41 }
```



主題三：

# 生命週期 (Lifecycle)



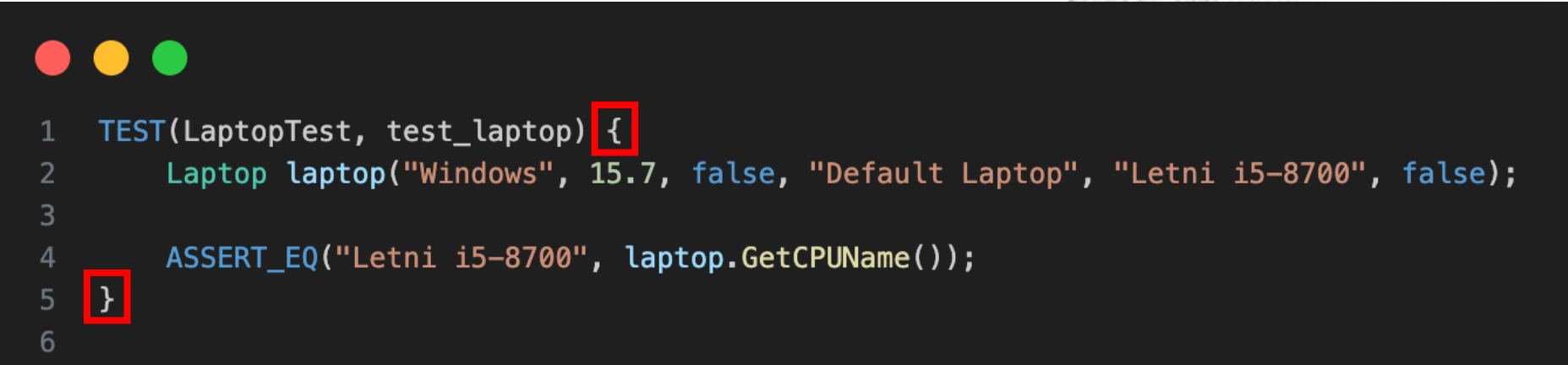
# 生命週期

- 對於一個物件來說，會有所謂的生命週期
- 當離開物件所在的**程式碼區塊**後，物件將會被釋放



# 程式碼區塊

- 什麼是程式碼區塊？



```
1  TEST(LaptopTest, test_laptop){
2      Laptop laptop("Windows", 15.7, false, "Default Laptop", "Letni i5-8700", false);
3
4      ASSERT_EQ("Letni i5-8700", laptop.GetCPUName());
5  }
6
```

- 對於程式碼來說，裡面的 laptop 在這個測試函數的程式區塊內
- 當離開這個程式區塊（例如測試結束）後，laptop 會被自動釋放
- Example

# 解構子

- 怎麼釋放?
  - 使用解構子來描述物件哪些資源要被釋放
  - 編譯器會自動呼叫 Laptop::~~Laptop()
- 如果使用物件時有以下動作時，會在解構子裡寫程式來 close 或 delete 它：
  - 開檔案
  - 開網路連線
  - 指標
  - 動態配置記憶體 (new)
- 否則會造成記憶體洩漏

```
1  #include "laptop.hpp"
2
3  #include <string>
4
5  Laptop::Laptop(std::string operatingSystem, double screenSize, bool isOLED,
6                std::string name, std::string cpuName, bool haveNumberKey){
7      this->operatingSystem = operatingSystem;
8      this->screenSize = screenSize;
9      this->isOLED = isOLED;
10     this->name = name;
11     this->cpuName = cpuName;
12     this->haveNumberKey = haveNumberKey;
13 }
14
15 Laptop::~~Laptop(){
16 }
17
18
19 std::string Laptop::GetOperatingSystem(){
20     return operatingSystem;
21 }
22
23 double Laptop::GetScreenSize(){
24     return screenSize;
25 }
26
27 bool Laptop::IsOLED(){
28     return isOLED;
29 }
30
31 std::string Laptop::GetName(){
32     return name;
33 }
34
35 std::string Laptop::GetCPUName(){
36     return cpuName;
37 }
38
39 bool Laptop::HaveNumberKey(){
40     return haveNumberKey;
41 }
```

# 生命週期

- 以生命週期來說：
  - 物件由建構子創立
  - 由解構子釋放



使用解構子釋放

```
1  #include "laptop.hpp"
2
3  #include <string>
4
5  Laptop::Laptop(std::string operatingSystem, double screenSize, bool isOLED,
6                std::string name, std::string cpuName, bool haveNumberKey){
7      this->operatingSystem = operatingSystem;
8      this->screenSize = screenSize;
9      this->isOLED = isOLED;
10     this->name = name;
11     this->cpuName = cpuName;
12     this->haveNumberKey = haveNumberKey;
13 }
14
15 Laptop::~Laptop(){
16 }
17
18
19 std::string Laptop::GetOperatingSystem(){
20     return operatingSystem;
21 }
22
23 double Laptop::GetScreenSize(){
24     return screenSize;
25 }
26
27 bool Laptop::IsOLED(){
28     return isOLED;
29 }
30
31 std::string Laptop::GetName(){
32     return name;
33 }
34
35 std::string Laptop::GetCPUName(){
36     return cpuName;
37 }
38
39 bool Laptop::HaveNumberKey(){
40     return haveNumberKey;
41 }
```

使用建構子創立





主題三：

# 資源取得即初始化

(Resource Acquisition Is Initialization, RAII)



# RAII (Resource Acquisition Is Initialization)

- 資源使用所經歷三個步驟
  - 資源獲取
  - 資源使用
  - 資源釋放 (Programmer 最常遺忘的環節)
- C++ 之父 (Bjarne Stroustrup) 提出了一個解決方案: RAII
  - 充分運用了 C++ 語言中區域物件自動銷毀的特性來控制資源的生命週期
- RAII 精神
  - 期望利用建構子初始化的物件是**好的**
  - 簡化許多異常行為的判斷
  - 與其在取值時檢查是不是合法的, 不如讓這個不合法的狀況不存在



Bjarne Stroustrup

# RAII - Bad Practice (1/2)

- 這段程式碼：
  - 期望飲料應該要有名字與大於零的容量
  - 若在取得時沒有名字與容量時，告訴使用者說沒有名字與容量
- 觀察：
  - 防止使用者輸出一些奇怪的情況
    - 在GetName()、GetVolume()檢常異常情況
  - 這還會發生什麼問題？
    - 只要每次用到這些值，就要額外再檢查一次
    - 使程式非常亂、不直觀

```
1  #include <string>
2
3  class Drink{
4  public:
5      std::string name;
6      int volume;
7
8      Drink(std::string name, int volume){
9          this->name = name;
10         this->volume = volume;
11     }
12
13     std::string GetName(){
14         if(name == ""){
15             throw std::string("No name");
16         }
17         return name;
18     }
19
20     int GetVolume(){
21         if(volume <= 0){
22             throw std::string("No volume");
23         }
24         return volume;
25     }
26 };
```

# RAII - Bad Practice (2/2)

- 如果這時候我們要增加一個新的函式來計算飲料價格
  - 計算的方法為容量 (cc) \* 0.03
  - Ex.  $1000 * 0.03 = 30$  (元)
- 可以發現，由於我們需要額外保證 volume 是好的
- 導致重複出現了需要判斷 volume 是否大於 0 的程式區塊

```
1  #include <string>
2
3  class Drink{
4  public:
5      std::string name;
6      int volume;
7
8      Drink(std::string name, int volume){
9          this->name = name;
10         this->volume = volume;
11     }
12
13     std::string GetName(){
14         if(name == ""){
15             throw std::string("No name");
16         }
17         return name;
18     }
19
20     int GetVolume(){
21         if(volume <= 0){
22             throw std::string("No volume");
23         }
24         return volume;
25     }
26
27     int GetPrice(){
28
29
30
31
32     }
33 };
```

# RAII - Good Practice

- 在建構子時即檢查並回饋使用者
  - 確保一開始所有的屬性都是使用合法的值
  - 未來就不需額外再次檢查
- 透過 RAII，我們可以確保物件內的成員都是使用合理的值。
- 透過不需額外判斷值是否合法而導致程式碼變得混亂。

```
1  #include <string>
2
3  class Drink{
4  public:
5      std::string name;
6      int volume;
7
8      Drink(std::string name, int volume){
9          if(name == ""){
10             throw std::string("No name");
11          }
12          if(volume <= 0){
13             throw std::string("No volume");
14          }
15          this->name = name;
16          this->volume = volume;
17      }
18
19      std::string GetName(){
20          return name;
21      }
22
23      int GetVolume(){
24          return volume;
25      }
26  };
```



# 物件導向程式設計

補充：標準模板庫

授課講師：孫勤昱博士

國立臺北科技大學 資訊工程系

[cysun@ntut.edu.tw](mailto:cysun@ntut.edu.tw)



# 大綱

- 什麼是 STL
- 實用 STL
  - `std::string`
  - `std::vector`
  - `std::shared_ptr`



# 什麼是 STL (1/3)

- Standard Template Library <sup>[1]</sup>
  - 中文翻譯為標準樣（模）板函式庫、泛型函式庫
  - 惠普實驗室開發，1988年成為國際標準，正式成為C++函式庫之重要成員
    - Alexander Stepanov



**Alexander Alexandrovich Stepanov** (俄語：Алекса́ндр Алекса́ндрович Степа́нов；1950年11月16日出生於莫斯科) 是一位俄裔美國電腦程式設計師，最著名的是標準程式設計人員的主要設計者者，<sup>[1]</sup>他於1992年左右在HP實驗室工作期間開始開發該軟體。早些時候，他曾在貝爾實驗室工作，與Andrew Koenig關係密切，並試圖說服Bjarne Stroustrup在C++中引入Ada泛型之類的東西。<sup>[2]</sup>他被譽為概念概念的提出者。<sup>[3][4]</sup>

他（與Paul McJones）是《程式設計元素》<sup>[5]</sup>的作者，該書源自Stepanov在Adobe Systems（受僱期間）教授的「程式設計基礎」課程<sup>[6]</sup>。他也與Daniel E. Rose合著了《從數學到通用編程<sup>[7]</sup>》一書。<sup>[7]</sup>他於2016年1月從A9.com退休。<sup>[8]</sup>

# 什麼是 STL (2/3)

- 它提供了多種模板類和函數，用於方便地操作資料結構和演算法
  - 容器 (Containers)：用來儲存資料的特殊結構，例如常見的vector, list, set, map等
  - 演算法 (Algorithms)：如排序、搜索、和資料操作等相關演算法
  - 迭代器 (Iterators)：用於訪問容器中的元素，例如begin(), end()等
  - 分配器 (Allocators)：用於定義如何分配和釋放記憶體物件
  - 配接器 (Adapters)：提供一種方式，使得我們可以在現有的容器（如 vector）上，用不同的方式操作這些容器
  - 函式對象 (Functors)：又稱為仿函數，使得演算法的行為可以根據需要進行自定義 (Override)
- C++的內建函式，方便我們撰寫code



# 什麼是 STL (3/3)

- 你可以用這些網站來取得 STL 的資源，認識一些有趣的 STL 來幫助你解決問題
  - <https://cplusplus.com/reference/>
  - <https://en.cppreference.com/w/>





主題一：

# 實用 STL

## `std::string` 與 `std::vector`



# 字串 (std::string)

- 用字串來儲存文字
- 透過字串，我們可以對文字進行處理
  - 例如取得字串長度、型態轉換、拼接

```
1  #include <iostream>
2  #include <string>
3
4  int main(){
5      std::string str = "Hello World"; // 宣告字串並賦值為 Hello World
6      std::cout << str << std::endl;  // 印出字串
7  }
```

# 取得字串 (std::string) 長度

- 使用 length() 函數來取得字串長度

```
1  #include <iostream>
2  #include <string>
3
4  int main(){
5      std::string str = "Hello World";    // 宣告字串並賦值為 Hello World
6      size_t length = str.length();      // 取得字串長度
7      std::cout << length << std::endl;  // 印出字串長度 ( 11 )
8  }
```

# 字串 (std::string) 拼接

- 我們可以使用運算子 “+” 來對任意兩個字串進行拼接

```
1  #include <string>
2
3  int main(){
4      std::string a = "Hello";
5      std::string b = "World!";
6      std::string c = a + b; // "HelloWorld!"
7  }
```

# std::vector

- 中文：**向量** Vector
  - 不是指數學上的向量，而是指一個可以動態調整大小的陣列（Array）
  - 動態記憶體陣列
    - `std::vector` 使用動態記憶體陣列，使陣列長度是可變化的
    - 比起之前需要分配記憶體來創建陣列（固定大小） Ex. `int a[5];`
- **基本上可以取代所有的陣列**
  - **安全：**使用 `std::vector` 通常比使用原始陣列更安全，因為不太可能超出其邊界。另外，它自動管理記憶體，減少了記憶體洩露的可能性。
  - **容易操作：** `std::vector` 提供了一系列的成員函式，如 `push_back()`, `pop_back()`, `size()`, `capacity()`, 和 `resize()`，這使得操作相對容易。



# std::vector

- 如何使用它?
- std::vector<int> 主要泛指這個動態記憶體陣列的型態
- 注意, std::vector使用大括號來存放元素
  - 非中括號 **[ ]**

```
1  #include <vector>
2
3  int main(){
4      std::vector<int> vec = {1, 2, 3, 4, 5};
5  }
```

# std::vector

- 如果我們有一個元素 6 想要接至 std::vector 的尾端,
- 我們可以使用 push\_back 來完成。

```
1  #include <vector>
2
3  int main(){
4      std::vector<int> vec = {1, 2, 3, 4, 5};
5      vec.push_back(6); // [1, 2, 3, 4, 5, 6]
6  }
```

# std::vector

- 取得陣列長度：使用 `size()` 來取得。

```
1  #include <vector>
2
3  int main(){
4      std::vector<int> vec = {1, 2, 3, 4, 5};
5      vec.push_back(6);    // [1, 2, 3, 4, 5, 6]
6      int size = vec.size(); // size = 6
7  }
```

主題二：

# 實用 STL

## std::shared\_ptr



# 為什麼我們需要 shared pointer?

- 以 C++ 來說，指標的釋放會是一個很大的議題，沒做好會導致 memory leak。





# 為什麼我們需要 shared pointer?

- 記憶體沒有管理好
  - Memory Leak 帶來的危害（實際課程案例）
    - 下學期的物件導向程式設計實習，如果沒有處理好 Memory Leak 的問題
    - 助教的電腦會藍屏，因為記憶體過度被占用導致電腦卡到無法處理其他程式
  - C/C++ 常發生的 Double free 跟 Use-After-Free 的安全漏洞
    - 安全程式設計
  - 70% 的 Security Bugs 來自於記憶體管理不當（微軟案例）<sup>[1]</sup>



[1] <https://www.zdnet.com/article/microsoft-70-percent-of-all-security-bugs-are-memory-safety-issues/>

# std::shared\_ptr

- 以**物件**的方式來處理指標
- 由於物件會隨著生命週期被釋放，因此以物件包裹的指標來說，我們可以仰賴生命週期來使用物件的方式將指標釋放，不用手動釋放或卡在 Memory Leak 地獄

```
1  template <typename T>
2  class SmartPtr {
3  public:
4      // 建構子，用於建立指標物件
5      SmartPtr(T *ptr = nullptr) { m_Ptr = ptr; }
6
7      // 解構子，用於釋放指標
8      ~SmartPtr() { delete m_Ptr; }
9
10     T GetValue() const { return *m_Ptr; }
11     void SetValue(T value) { *m_Ptr = value; }
12
13 private:
14     T *m_Ptr;
15 };
16
```

# std::shared\_ptr

物件建立



建立指標物件

離開物件程式區塊 (物件釋放)

先釋放指標



後釋放物件



```
1  template <typename T>
2  class SmartPtr {
3  public:
4      // 建構子，用於建立指標物件
5      SmartPtr(T *ptr = nullptr) { m_Ptr = ptr; }
6
7      // 解構子，用於釋放指標
8      ~SmartPtr() { delete m_Ptr; }
9
10     T GetValue() const { return *m_Ptr; }
11     void SetValue(T value) { *m_Ptr = value; }
12
13 private:
14     T *m_Ptr;
15 };
16
```

# 為什麼我們需要 shared pointer?

- 用這一個方式可以不用手動釋放記憶體空間
- 將所有的指標視同物件使用
- 使記憶體管理更加方便

